
TCH057 — APPLICATIONS MOBILES ET EXPÉRIENCE USAGER

FitZone

Rapport du projet de session

Équipe : Bryan Chedjou, Ramzi [nom de famille], Mike Salomon

Session : Été 2026

Date : 12 août 2026

Dépôt : github.com/Waxwell-Agency/FitZone

SECTION 1

Description de l'application

FitZone est une application mobile Android destinée aux clients d'un centre d'entraînement. Elle permet à un usager de créer un compte et de se connecter, de consulter les programmes d'entraînement auxquels il est inscrit, de suivre ses séances et leurs échéances, de répondre à des quiz de connaissances liés à ses programmes, de consulter des conseils nutritionnels adaptés à la pratique de l'activité physique, et de gérer les informations de son profil.

L'objectif principal est de centraliser le suivi de la progression de l'utilisateur dans une interface simple, cohérente et facilement consultable sur téléphone. La conception respecte les principes ergonomiques vus en cours (heuristiques de Nielsen, critères de Bastien et Scapin) : cohérence visuelle entre les écrans, retour d'information explicite (messages d'erreur clairs, indicateurs de statut visuels), guidage et navigation prévisible via une barre de navigation inférieure fixe présente sur les écrans principaux.

SECTION 2

Principaux choix de conception

Architecture et organisation du code

Le code est organisé en packages selon l'architecture standard enseignée dans le cours, sous le package de base `com.ramzi_mike_bryan.FitZone` :

- `activites` — les écrans (Activities) de l'application ;
- `adapters` — les adaptateurs RecyclerView pour les listes dynamiques (programmes, séances, quiz, aliments) ;
- `dao` — l'accès aux données locales (base SQLite, pour l'enregistrement des résultats de quiz) ;
- `com.models` (package partagé) — les classes de modèle reflétant la structure du serveur JSON (User, Programme, Seance, Quiz, Aliment, QuizResult) ;
- `com.api` (package partagé) — la couche d'accès réseau, avec l'interface `ApiService` et la classe `RetrofitClient` .

Navigation entre les écrans

La navigation principale se fait par une barre inférieure fixe à cinq destinations (Accueil, Programmes, Séances, Quiz, Profil), présente sur les écrans de premier niveau. Les écrans secondaires — Détail d'un programme, Passation d'un quiz, Conseils nutritionnels — sont accessibles depuis un écran parent et utilisent une flèche de retour plutôt que la barre de navigation, afin de signaler clairement qu'il s'agit d'une sous-page plutôt que d'une section principale.

Accès aux données

Les données (utilisateurs, programmes, séances, quiz, aliments) sont servies par un serveur JSON local (`json-server`) à partir du fichier `fitzone.json` . Les appels réseau passent par Retrofit avec un convertisseur Gson. Les résultats de quiz sont en plus enregistrés localement dans une base SQLite (via la classe `QuizResultDao`), afin de conserver un historique consultable hors ligne, conformément à l'énoncé du projet.

Interface utilisateur

L'interface reprend un système visuel cohérent sur l'ensemble des écrans : une couleur d'accent violette, des cartes blanches à coins arrondis avec une bordure légère, des

indicateurs de statut sous forme de badges colorés (à faire, en retard, validé, terminé), et une bannière en dégradé pour les écrans de résumé (Accueil, détail d'un programme). Les maquettes d'interface ont été produites à partir de ce système visuel afin de rester cohérentes avec l'application développée.

SECTION 3

État du projet

Le projet est actuellement **partiellement fonctionnel**.

Fonctionnalités complètes

- Connexion et inscription, avec validation des identifiants auprès du serveur ;
- Consultation et modification du profil utilisateur ;
- Consultation des programmes inscrits, avec recherche et filtrage ;
- Consultation du détail d'un programme (description, coach, annonces) ;
- Consultation des séances d'un programme, avec statut et échéance ;
- Tableau de bord (Accueil) : résumé du statut, programmes, quiz et séances à venir, annonces récentes ;
- Quiz : consultation de la liste, passation question par question, calcul et enregistrement local du score ;
- Conseils nutritionnels : liste des aliments recommandés avec calories et moment de consommation suggéré.

Fonctionnalités incomplètes ou non implémentées

- **Soumission d'une séance** : les séances peuvent être consultées, mais la soumission simulée décrite dans l'énoncé (saisie d'un lien ou d'un texte, passage du statut à « Soumise ») n'est pas encore implémentée à ce jour.
- L'écran Profil permet de modifier l'URL de la photo, mais n'affiche pas encore l'image correspondante.

Problèmes rencontrés

- **Adresse du serveur codée en dur**. L'application pointait initialement vers l'adresse IP locale d'un membre de l'équipe (`RetrofitClient.java`), ce qui empêchait les autres membres — et un correcteur externe — de se connecter au serveur sans modifier le code. Cette limitation a été corrigée en exposant le serveur via un tunnel public, accessible sans configuration locale par tous les membres de l'équipe.
- Plusieurs conflits de fusion Git sont survenus lorsque deux membres modifiaient le même fichier au même endroit — notamment l'interface `ApiService`, à laquelle chaque membre ajoutait ses propres routes en parallèle. Ces conflits ont été résolus manuellement, sans perte de travail.

- Une classe `QuizResult` a été créée en double, dans deux contextes différents : une version côté serveur (`com.models` , reflétant les données JSON) et une version locale pour la persistance SQLite (`com.ramzi_mike_bryan.FitZone.dao`). Les deux cohabitent sans conflit de compilation grâce à des packages distincts, mais mériteraient d'être clarifiées dans une itération future.

SECTION 4

Informations sur l'environnement de développement

Langage	Java (compatibilité Java 11)
SDK Android (compileSdk)	36, révision 1
SDK minimum (minSdk)	29
SDK cible (targetSdk)	36
Android Gradle Plugin (AGP)	9.2.0
Gradle	9.5.0
Bibliothèques principales	Retrofit 2.11.0 (+ convertisseur Gson), AndroidX AppCompat 1.7.1, Material Components 1.14.0, ConstraintLayout 2.2.1
Serveur de données	json-server (Node.js), à partir du fichier <code>fitzone.json</code>
Base de données locale	SQLite (résultats de quiz)

Le fichier `local.properties` (chemin du SDK Android) est propre à chaque poste de développement et n'est pas versionné dans le dépôt Git, conformément aux bonnes pratiques Android.